

Document number: P1479R0

Date: 2019-01-21

Reply-to: Robert Kawulak <Robert.Kawulak@gmail.com>

Audience: SG18 (LEWG Incubator)

ostream wrapper

I. Table of Contents

- II. [Introduction](#)
- III. [Motivation](#)
- IV. [Q&A](#)
- V. [Open Issues](#)
- VI. [Impact on the Standard](#)
- VII. [Technical Specifications](#)

II. Introduction

This paper proposes adding a standard library function that wraps the common idiom of converting an object to a string using an ostream:

```
ostream os;  
os << obj;  
string text = os.str();
```

The proposed solution is:

```
template<class charT, class traits = char_traits<charT>,  
        class Allocator = allocator<charT>, class... Args>  
basic_string<charT,traits,Allocator> to_basic_string(Args&&... args)  
{  
    basic_ostream<charT,traits,Allocator> stream;  
    stream.exceptions(ios_base::failbit | ios_base::badbit);  
    (stream << ... << forward<Args>(args));  
    return move(stream).str(); // make use of P0408  
}  
  
template<class... Args>  
string to_string(Args&&... args)  
{  
    return to_basic_string<char>(forward<Args>(args)...);  
}  
  
// other variants like to_wstring etc. (omitted for brevity)
```

Note that the name `to_string` may be treated as *tentative* for a lack of a better alternative for now and is possibly a subject of future bikeshedding (see [Naming](#)).

This paper is a follow-up to [P0117 'Generic to_string/to_wstring functions'](#), which was discussed at the Kona 2015 committee meeting. The author was not present at the meeting, but from the [feedback received](#) he

concludes that the paper hasn't been motivated well enough and possibly the intentions were misunderstood. In this (completely rewritten) paper, the author tries to clarify the [motivation](#) and [address the feedback](#).

III. Motivation

Using `ostringstream` to convert an object to its text representation is a familiar programming pattern in C++. The problem with using it is its verbosity – something that ought to be a single function call is most commonly a three-line boilerplate code injecting unnecessary variables into the scope and often ignoring error handling. While C++11 somehow improved the situation by providing a set of overloaded `to_string` functions, these are only limited to built-in numeric types.

The main features of the proposed solution are:

- it wraps the common idiom in a simple and easy-to-use interface,
- doesn't introduce any new concepts or formatting APIs, therefore:
 - is trivial to learn by anyone familiar with `iostreams`,
 - takes advantage of existing `iostream` support for user defined types (including [manipulators](#)),
- makes it harder to ignore errors by enabling exceptions for the used stream,
- avoids accidentally copying the result rather than moving it,
- is generalised into a [variadic function](#) to improve its usability.

The proposed function is also [mostly consistent](#) with the `to_string` functions already found in the Standard Library and could act as a generalisation of those, making them optimised special cases. However, this is not the main point of this proposal and the utility can be added as an unrelated function [with a different name](#) as well.

A few examples:

```
// a simple case:

complex c{0, 1};
assert(to_string(c) == "(0,1)");

// easy concatenation:

minutes m{10};
assert(to_string(m.count(), "min") == "10min");

// manipulators work too:

bitset<3> b{0b010};
assert(to_string(setfill('0'), setw(6), b) == "000010");

// a user-defined type with existing elaborate iostreams support:

struct coord { int x, y; };
ostream& operator<<(ostream&, coord);
ostream& coord_labelled(ostream&); // manipulator setting "labelled" format

coord c{2,4};
assert(to_string(c) == "[2,4]");
assert(to_string(coord_labelled, c) == "[x=2,y=4]");
```

IV. Q&A

Are there any customisation points for user-defined types?

Yes – the solution reuses the most common customisation point for representing objects as text in C++: a stream insertion operator. While not the most efficient, this interface is well-known and widespread. Why not go further and provide a second customisation point to get more efficiency? Because:

- the crucial aspect of this proposal is simplicity – adding more customisation points would complicate the design considerably. Sacrificing the simplicity isn't necessarily worth the perceived performance gain;
- if one really cares about performance, then `to_string` returning a dynamically-allocating string is possibly already a too heavy-weight API (that's why the `to_chars/from_chars` functions were added despite already having `to_string` for numeric types);
- for the use-cases when efficiency has a high priority, [P0645 'Text Formatting'](#) will provide an API for custom type conversions avoiding streams, locales and dynamic allocations.

Isn't this redundant with [P0645 'Text Formatting'](#)?

This proposal (unlike P0645) doesn't really introduce anything like a new formatting API – it only encapsulates the common iostreams use pattern, and nothing more. While certainly there is some overlap, `to_string` is more direct in basic use cases in which the required format string would simply be redundant, for instance:

```
string s = to_string(obj);
```

versus somewhat clumsy:

```
string s = format("{} ", obj);
```

Moreover, an important advantage of `to_string` is its compatibility with any already existing user-defined stream manipulators altering the formatting, while in order to use P0645 the user would have to write analogous formatters from scratch.

To recap, the two mechanisms can live side-by-side being complementary – `to_string` being useful for simple use cases and for types that already have an elaborate iostream formatting support implemented, while P0645 being good for more involved needs and for users willing to write formatters conforming to the new API.

Why a variadic function rather than a single-argument one?

Simply because it is a low-hanging fruit that doesn't cost much in terms of simplicity or otherwise, yet provides significant gains in functionality. Apart from allowing the use of stream manipulators, it provides a more concise alternative to the following:

```
string s = to_string(obj1) + to_string(obj2) + to_string(obj3);
```

which can be written as:

```
string s = to_string(obj1, obj2, obj3);
```

The usability gains of the variadic version shouldn't be underestimated. The author has some experience with a very similar utility in a codebase he's working on professionally and gathered some numbers to illustrate this point. Although – statistically speaking – the sample size is rather small, the proportions clearly show the practical usefulness of the variadic version:

- all calls in the code: 12;
- single-argument vs. multiple-argument calls: 2 vs. 10;
- calls using stream manipulators: 3;

- calls taking advantage of multiple arguments concatenation: 8.

Isn't supporting stream manipulators too clever?

There's nothing magical or tricky done here to make manipulators supported and the author doubts that anyone familiar with iostreams would have trouble understanding how this utility works, including manipulators. In fact, artificially inhibiting manipulators use would make the utility much more complicated to implement and, arguably, surprisingly inconsistent.

V. Open Issues

Naming

While `to_string` is not a bad candidate for the name of the utility, being concise, correct about what it does and fitting as a generalisation of the already existing `to_string` functions, it's not perfect either: it doesn't hint at using iostreams as the underlying mechanism and may cause a breaking change in rare cases (see [Impact on the Standard](#)). Some other ideas for the name are:

- `to_stringstream`
- `to_stream`
- `to_text`
- `stringize`
- `streamify`
- `lexify`

The author is open to other naming proposals.

Analogous parsing functionality

While the paper is concerned with conversion of objects to strings, it is tempting to provide a symmetrical utility for easy parsing of strings as objects. Therefore it is shown here for consideration of the committee and will be included in a future revision of this proposal if the committee is in favour of it. It could look like this:

```
template<class charT, class traits = char_traits<charT>, class... Args>
streamsize from_basic_string(basic_string_view<charT,traits> view, Args&&... args)
{
    basic_istream<charT,traits> stream{span<charT>{view}}; // see notes below
    stream.exceptions(ios_base::failbit | ios_base::badbit);
    (stream >> ... >> forward<Args>(args));
    return view.length() - stream.tellg(); // see notes below
}

template<class... Args>
streamsize from_string(string_view view, Args&&... args)
{
    return from_basic_string(view, forward<Args>(args)...);
}

// other variants like from_wstring etc. (omitted for brevity)
```

There are two things to note here. First, rather than a `basic_string`, the function takes a `basic_string_view` as its argument. This makes this function more generic and callable not only with string arguments, but also with

string splices or C-style strings. Also, the stream type used here is `basic_istream` (proposed in [P0448](#)) rather than `basic_istream` to avoid copying of the input data unnecessarily.

Second, it is an open question what to do in the situation when the input has been consumed only partially. One option is to throw an exception, another one is to signal this with a return value – either just a boolean, or e.g. the number of bytes left unconsumed. The code assumes the last option for the sake of example, but the author doesn't have a strong opinion here.

VI. Impact on the Standard

Generally speaking, the proposal describes a new component of the Standard Library which should have no impact on existing code.

However, if the proposed function template is called `to_string`, it will overload existing Standard Library functions with this name. In large majority of cases the result of the new function is consistent with the existing functions, but in some obscure corner cases it's not, yielding a change of behaviour. Specifically, these cases are calls of `to_string` with an argument that doesn't match a parameter type of any of the existing overloads, but is implicitly convertible to one of the types. For example, the call `to_string('0')`, which was resolving to the `to_string(int)` overload and yielded `"48"` (assuming ASCII encoding) would now call the new function template's specialisation `to_string<char>(char&&)` yielding `"0"` (which is actually a more reasonable outcome, but a breaking change nonetheless).

VII. Technical Specifications

Proposed wording will be provided in a future revision of the document.